1. Организация

2. Преговор
2.1 Упражнение 1

3. Какво е операционна система?
3.1 Упражнение 2

Занятия

- Два блока седмично; във всеки от тях един учебен час теория и един учебен час практика.
- За практическите занятия Ви е необходим *компютър*. В случай, че за някой конкретен това не е така, ще предупреждавам своевременно.

Конспект - сбита версия

0. Въведение в ОС
1. Програмиране на C
2. Памет
3. Процесор
4. Файлови системи
5. Мрежи
6. Сигурност

Изисквания

- *Писане* - за всеки блок на случаен принцип избираме *писар*, който да състави подробни и ясни записки за съответното занятие. Ще предоставя шаблон, в който да вместите Вашите бележки.
- *„Малки“ контролни* - върху всяка точка от сбития конспект; провеждани в час за не повече от 40 минути. Ще предупреждавам минимум две седмици преди всяко контролно, но очакваните дати са за средите на октомври, ноември и декември.

Изисквания

- *„Големи“ контролни* - върху няколко точки от сбития конспект. След Коледа.
- *Домашни* - скромни и с упражнителна цел; писане на код. След проверка на домашни ще се провеждат устни защити с част от предалите. Не мамете.
- *Проект* - по-нескромна задача за писане на код. Приветстват се собствени теми, но те трябва да бъдат предварително одобрени.

Материали

- Считайте презентациите, които ще осигурявам за основен източник.
- Другото най-полезно четиво са записките, направени от Вас.
- Книги (неизчерпателен списък) -
  - Operating Systems: Three Easy Pieces - https://pages.cs.wisc.edu/~remzi/OSTEP/
  - xv6 - https://pdos.csail.mit.edu/6.828/2018/xv6/book-rev10.pdf
  - Modern Operating Systems, Tanenbaum
  - The Design of the UNIX Operating System, Bach
  - The C Programming Language, Ritchie
  - Advanced Programming in the UNIX Environment, Stevens

Материали

  - Expert C Programming: Deep C Secrets, Linden
- Справочници (неизчерпателен списък) -
  - `man` страниците - или командата, или `https://www.man7.org/linux/man-pages`
  - Документацията на Intel - `https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html`
  - и други, които ще Ви посочвам, когато ще са полезни.

Бройни системи

Какво е бройна система?

$$14_{(10)} = 1 \cdot 10^1 + 4 \cdot 10^0 \implies 14_{(10)}$$
$$14_{(10)} = 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 \implies 1110_{(2)}$$
$$14_{(10)} = 1 \cdot 3^2 + 1 \cdot 3^1 + 2 \cdot 3^0 \implies 112_{(3)}$$
$$14_{(10)} = 1 \cdot 8^1 + 6 \cdot 8^0 \implies 16_{(8)}$$
$$14_{(10)} = 14 \cdot 16^0 \implies 14_{(16)} \equiv E_{(16)}$$

Бройни системи

$$14_{(10)} = 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 1110_{(2)}$$

Бройни системи

$$14_{(10)} = 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 1110_{(2)}$$

База

Бройни системи

$$14_{(10)} = 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 1110_{(2)}$$

Коефициенти

Бройни системи

$$14_{(10)} = 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 1110_{(2)}$$

Позиции

Бройни системи

$$14_{(10)} = 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 1110_{(2)}$$

База

Коефициенти

Позиции

От значение е и базата на първоначалната бройна система база.

Бройни системи

- Най-важните бройни системи за нас са двоична и шестнайсетична.
- Първата, заради нейната тясна връзка с хардуера, върху който се изпълняват нашите програми. Ще бележим с префикс `0b`.
- Втората, защото е много лесно преобразуването ѝ към двоична и обратно. *Защо?* Ще бележим с префикс `0x`.

Бройни системи

- Ключовото наблюдение е, че една шестнайсетична цифра описва точно половин байт - т.е. четири бита. Най-голямата шестнайсетична цифра е `f`.

$$\texttt{f}_{(16)} = 1111_{(2)}$$
$$\texttt{ff}_{(16)} = 11111111_{(2)}$$

Бройни системи

- Превръщаме от двоична към шестнайсетична, разделяйки поредицата от битове (започвайки от най-младшия) на групи от по четири (наричаме ги тетрада), и на всяка от получените групи съпоставяме съответната шестнайсетична цифра.
- Обратната посока. Превръщаме от шестнайсетична към двоична, като на всяка цифра съпоставяме група от по четири бита.

Няколко важни константи

Уговорка. За нас един килобайт е 1024 байта. Аналогично имаме

$$1\,\mathrm{KB} = 1024\,\mathrm{B}$$
$$1\,\mathrm{MB} = 1024\,\mathrm{KB}$$
$$1\,\mathrm{GB} = 1024\,\mathrm{MB}$$
$$1\,\mathrm{TB} = 1024\,\mathrm{GB}$$
$$1\,\mathrm{PB} = 1024\,\mathrm{TB}$$

Няколко важни константи

Няма да се придържаме към системата SI, която определя един килобайт като 1000 байта, а 1024 байта нарича кибибайт. Важните константи са:

$$1\,\mathrm{KB} = 2^{10}\,\mathrm{B}$$
$$1\,\mathrm{MB} = 2^{20}\,\mathrm{B}$$
$$1\,\mathrm{GB} = 2^{30}\,\mathrm{B}$$
$$1\,\mathrm{TB} = 2^{40}\,\mathrm{B}$$
$$1\,\mathrm{PB} = 2^{50}\,\mathrm{B}$$

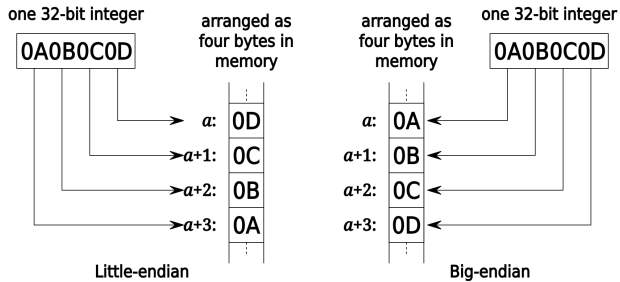
Няколко важни константи

Тяхното помнене е доста удобно, когато трябва бързо да превърнете произволно число - било то отместване във файл или големина на описвано множество от побитова карта, в размер.

Индианци

- *Пример.* Нека имаме целочислено число, съставено от четири байта (`uint32_t`). В паметта то се представя чрез поредица от съседни байтове - това е неговият *запис*. Програмистът влага на тези байтове *смисъл* на число, представено в двоична бройна система.
- За реда на байтовете (записа) има два общоприети варианта - отдясно-наляво или отляво-надясно; т.е. младшите битове се намират съответно или най-вдясно, или най-вляво.

Индианци



Фигура: Илюстрация на endianness при 4-байтово число.

Индианци

- Когато байтовете на данните са разположени отдясно-наляво, казваме, че се използва big endian. Обратно, ако са отляво-надясно - little endian.[1]
- При повечето архитектури (x86, ARM) се използва little-endian.
- При комуникация през мрежата се използва big-endian (още наричан network order).

[1] Терминът *endianness* произлиза от творбата „Пътешествията на Гъливер“ на Джонатан Суифт, в един епизод от която две групи лилипути имат спор за това от кой край трябва да се чупи сварено яйце - дали от малкия (little end), или от големия (big end). Оттам идват и съответно little-endian и big-endian. Действителна връзка с индианци няма. ☺

Упражнение

- Превърнете 0b111011110011 в шестнайсетична бройна система. Отг. 0xef3
- Ако горното число беше размер на файл, то колко щеше да бъде голям той? Отг. 3KB 755B
- Ако горното число беше записът в little-endian, то колко щеше да бъде размерът на файла? Отг. 0b001111111110 = 0x3fe = 1022B
- 0x8000000, 0x1000, 0x400, 0x10000. Отг. 128MB, 4KB, 1KB, 64KB
- Използвайте програма, написана на C, за да проверите дали машината Ви използва little- или big-endian за 4-байтово целочислено число. *Подсказка.* В gdb използвайте команда `x/4b &a`, където a е вашата променлива.

Видове софтуер

Какво е системно програмиране?

Видове софтуер

Какво е системно програмиране?

Компонент	**Интерфейс**	**Функции**
Бизнес модул	Към потребителя - конзола, графично	„Бизнес“
Библиотеки за общо предназначение	Прост	Общи - низове, памет, математика и т.н.
Библиотеки за достъп до хардуер	Логическо устройство	Управление на конкретен хардуер

Какво е операционна система?

- Операционната система (ОС) е софтуер, който споделя и разпределя ресурсите на хардуера между различни програми.
- Тези програми получават възможност от ОС да не се съобразяват с останалите агенти на системата, а само със самата ОС.
- Споделянето на ресурсите се осъществява чрез *изолация* и/или чрез *заявки*.
- *Изолацията* е метод, чрез който ОС показва даден ресурс само и единствено на неговия *собственик*.

Какво е операционна система?

- *Методът на заявките* изисква изрично обръщения на всички програми към един и същ ресурс едновременно. Този вид споделяне бива два вида - *последователно* (или блокиращо, или синхронно) и *паралелно* (или асинхронно). При първото програмата блокира, докато не завърши изпратената заявка, докато при второто заявката се поставя в опашка за изпълнение и програмата няма контрол в какъв момент точно ще бъде изпълнена.

Какво е операционна система?

Ресурс	**Абстракция**	**Метод на споделяне**
Процесор	Нишки	Изолация (и времеделене)
Памет	Адресно пространство	Изолация
Диск	Файлова система	Заявки
Мрежа	Сокети	Заявки
Принтер	Логически принтер	Заявки (асинхронни)

Какво е операционна система?

Изискванията към една ОС още включват -

- Скорост - цената за изпълнение на ОС трябва да бъде минимална.
- Надеждност - от издръжливостта на ОС зависят и приложенията.
- Изолация между изпълняващите се приложения с цел сигурност. Същевременно, начин за споделяне на данни между приложения с цел кооперация.
- Абстрахиране на конкретния хардуер с цел преносимост.
- Удобен интерфейс, за да бъде лесна за употреба.

Структура на ОС

След като сме описали задачите, които една ОС трябва да решава, нека опитаме да си представим как изглежда горе-долу една такава.

Нека разгледаме един крайно опростен пример. Имаме приложения, които искат да пишат по диска, за да запазват потребителски данни. ОС, в която се изпълняват, предоставя интерфейс за работа с диска - приложенията извикват `disk_write(sector, data)` и ОС се грижи за магията.

Имаме две непосредствени следствия-проблеми. От една страна това означава, че приложение A може да реши да

Структура на ОС

използва сектор 42 и приложение Б да си хареса същия сектор. Така те ще бъдат в постоянно състояние на *състезание* (на англ. race condition) за данните на сектор 42.

От втора страна, щом това е просто функция, която може да се извика от всеки, то системата е много лесна за компрометиране - произволна програма може да изтрие данните на потребителя. Със сигурност нежелан ефект!

Достигаме до много важен извод - ОС **не** е просто библиотека с функции, които се „грижат за хардуера“. *Откъде точно произлиза разликата обаче?*

Структура на ОС

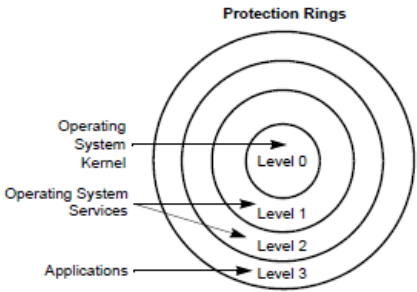
Разликата е, че не всяка програма трябва да може да пише по диска каквото и където си пожелае. Това трябва да бъде допустимо при силни ограничения - все пак не бива да го забраняваме; дискът наистина е за писане. *Как решаваме този проблем?*

Понеже разбираме, че трябва дискът да бъде споделен между множество програми, си спомняме, че това се случва чрез *заявки* към *абстракцията* на диска, а именно - *файловата система*. Така решаваме единия проблем.

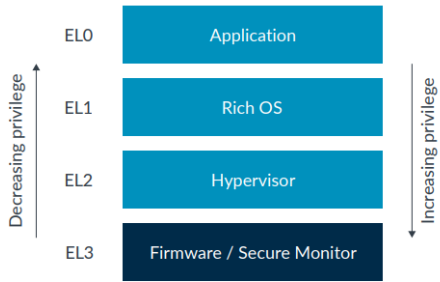
Структура на ОС

Вторият проблем, който представихме, се отнася до сигурността на ОС. Решаваме го по следния начин. Въвеждаме понятието ниво на привилегия (на англ. privilege level или още protection rings). Това е свойство на изпълняваща се програма - казваме, че се изпълнява с някакво ниво на привилегия.

Структура на ОС



Фигура: x86 Нива на привилегия



Фигура: ARM Нива за изключения

Структура на ОС

И тогава решението на проблема ни е много просто - за да изпълнява `disk_write()` (от предходния ни пример), изискваме програмата да се изпълнява с ниво на привилегия, което позволява това.

Както уточнихме вече обаче приложенията не се изпълняват с ниво на привилегия, което позволява това. И това е по дизайн.

Нуждаем се от метод за „промяна на нивото на привилегия“ - изпълняваме си приложението, вършейки позволени операции (математически операции, работа с низове, взаимодействие с потребител), и в някакъв момент искаме да извикаме

Структура на ОС

„забранената“ функция `disk_write()`. След като функцията приключи работата си, искаме нивото на привилегия да се смени отново - този път към началното си състояние.

Този механизъм се нарича системно извикване (на англ. system call или накратко syscall). Чрез него потребителско приложение инициира работа за ОС. Казваме, че приложението се изпълнява в потребителски режим (на англ. user mode), а ОС - в режим на ядрото (на англ. kernel mode). Онази част от ОС, която се изпълнява в режим на ядрото, наричаме ядро (на англ. kernel). При изпълнение в режим на ядрото програмата има

Структура на ОС

достъп до цялата системна памет и всички инструкции на процесора, докато при потребителския режим ОС се налагат ограничения.

Нека имаме едно примерно системно извикване `S()`. Технологията на извикване на `S()` е следната. Когато потребителско приложение A желае да изпълни `S()`, то трябва да задейства софтуерно прекъсване „капан“ (на англ. trap software interrupt), при което процесорът ще съхрани състоянието на A, ще премине към режим на ядрото и ще премине към изпълнение на обработка на софтуерното прекъсване. При

Структура на ОС

обработката на софтуерното прекъсване ще се изпълни работата на `S()`, след което процесорът ще изпълни специална инструкция за излизане от капана, ще възвърне състоянието на приложението и то ще продължи работата си.

Забелязваме обаче един важен детайл. При извикването на `S()` не можем просто да посочим неговия адрес. *Защо?*

Например, ако желаем да отворим файл, за който нямаме права, можем да посочим адрес, съответстващ на функцията след проверката за правоспособност. Така получаваме способност за отваряне на произволен файл, заобикаляйки

Структура на ОС

всякакви проверки за сигурност. Това се нарича уязвимост (на англ. vulnerability).

Решението на този проблем е просто. Ядрото използва фиксирана таблица за своите системни извиквания. Потребителско приложение уточнява единствено индекса на системното извикване, което иска да се изпълни, а ядрото намира адреса на съответстващата функция при обработката на софтуерното прекъсване.

Така изяснихме общата постановка на комуникация между ОС и приложенията, които ОС управлява.

Структура на ОС

Примери за системни извиквания в Linux са `write`, `mmap`, `shmat` и `dup`. Общо са около 500. Може да се запознаете по-обстойно с всеки от тях, използвайки командата `man 2 <syscall>`. Можете да разгледате системните извиквания, които Linux ядрото предоставя, търсейки макрото `SYSCALL_DEFINE`*x*, където *x* е цяло число от 0 до 6 (броя параметри) в хранилището на кода - torvalds/linux.git или в индексиращата система Bootlin. Да отбележим, че когато извикаме например `write` в своя програма това **не** е системното извикване, както очакваме. Всъщност това е функция, дефинирана в стандартната

Структура на ОС

библиотека `glibc`, която извършва описания от нас процес с поставянето на число на определено място и предизвикване на софтуерно прекъсване. Такива функции се наричат обвивка (на англ. wrapper или jacket).

В MacOS ситуацията е подобна на тази в Linux, поради споделения им произход. Можете да разгледате системните извиквания в хранилището на кода - apple/xnu.git. Там вместо подход с макроси, използват генератор на код.

В Windows обичайно се използват различни имена на понятията, но фундаментално идеята е същата. Наместо

Структура на ОС

системно извикване, популярният термин е системна услуга (на англ. system service). Технологията на извикване е сходна - числото, подадено от извикващия се използва от *диспечера на системни услуги*, за да индексира таблица със самите функции, имплементиращи системното извикване. Самите системни извиквания са недокументирани официално, но до известна степен следват т.нар. Native API, имплементиран в `NTDLL.dll`, което е „най-ниската точка“, изпълняваща се в потребителски режим. Примери за системни услуги в Windows са `NtReadFile` и `NtWaitForSingleObject`.

Упражнение

Забележка: Понеже все още не сме преминали през темите относно C, задачките ще изискват само най-елементарни познания и вярвам, че ще се справите ☺.

- Напишете програма, която приема от стандартния си вход три аргумента - име на файл, отместване във файла и низ. Използвайки обвивките на системните извиквания за работа с файл - ОТВАРЯНЕ, ПИСАНЕ и ЗАТВАРЯНЕ, програмата записва на посоченото отместване данните във файла и приключва.

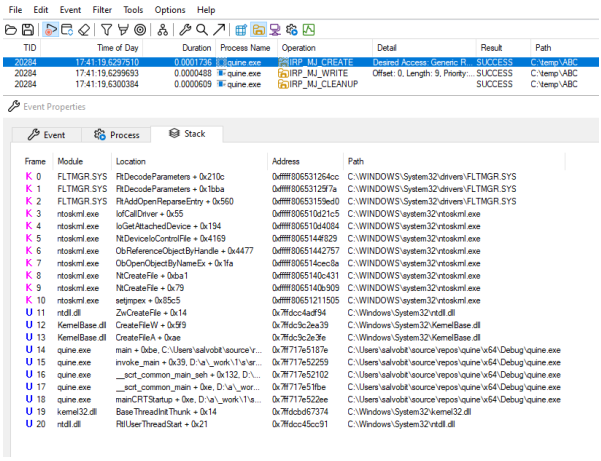
Отг. Под Linux и Mac OS използвайте open, write и close, а под Windows - CreateFileA, WriteFile и CloseHandle.

Упражнение

- Използвайте програма за проследяване на системните извиквания, за да разгледате хода на програмата си. В Windows използвайте `Procmon`, в Mac OS - `sudo dtruss`, а в Linux `strace`. Ако имате други предпочитания, нямам против.
- *Забележка (Windows):* Това, което се вижда в колоната „Operation“ в `Procmon` не са системните услуги. Същевременно при *<десен клик>* $\rightarrow$ *Stacks* се виждат ясно всички извиквания, които се пораждат от функцията, както и това в какъв момент точно се преминава към режим на ядрото.

```
1377/0x3ebc:      3829       2      1 mprotect(0x104AEC000, 0xC8, 0x1)     = 0 0
1377/0x3ebc:      3831       2      0 mprotect(0x104AFD000, 0x1000, 0x3)     = 0 0
1377/0x3ebc:      3834       2      0 mprotect(0x104AFD000, 0x1000, 0x1)     = 0 0
1377/0x3ebc:      4007       2      0 issetugid(0x0, 0x0, 0x0)     = 0 0
1377/0x3ebc:      4336       6      3 getentropy(0x7FF7BB409510, 0x20, 0x0)     = 0 0
1377/0x3ebc:      4463      15     12 getattrlist("/Users/salvobit/Desktop/a.out\0", 0x7FF7BB409CB0, 0x7FF7BB409CCC)     = 0 0
1377/0x3ebc:      4483      18     17 access("/Users/salvobit/Desktop\0", 0x4, 0x0)     = 0 0
1377/0x3ebc:      4497      14     12 open("/Users/salvobit/Desktop\0", 0x0, 0x0)     = 3 0
1377/0x3ebc:      4500       3      1 fstat64(0x3, 0x7F90A7704490, 0x0)     = 0 0
1377/0x3ebc:      4502       2      0 csrctl(0x0, 0x7FF7BB409ECC, 0x4)     = 0 0
1377/0x3ebc:      4504       3      1 fcntl(0x3, 0x32, 0x7FF7BB409BC0)     = 0 0
1377/0x3ebc:      4507       2      1 close(0x3)     = 0 0
1377/0x3ebc:      4521      10      8 open("/Users/salvobit/Desktop/Info.plist\0", 0x0, 0x0)     = -1 Err#2
1377/0x3ebc:      4539       3      1 proc_info(0x2, 0x561, 0xD)     = 64 0
1377/0x3ebc:      4555       3      1 csops_audittoken(0x561, 0x10, 0x7FF7BB409F10)     = -1 Err#22
1377/0x3ebc:      4571      10      7 sysctl([unknown, 3, 0, 0, 0, 0] (2), 0x7FF7BB40A250, 0x7FF7BB40A248, 0x7FF813739BA5, 0x15)     = 0 0
1377/0x3ebc:      4574       3      1 sysctl([CTL_KERN, 137, 0, 0, 0, 0] (2), 0x7FF7BB40A2E0, 0x7FF7BB40A2D8, 0x0, 0x0)     = 0 0
1377/0x3ebc:      4761     349    138 open("/tmp/ABC\0", 0x602, 0xDAA8)     = 3 0
1377/0x3ebc:      4781      20     18 write(0x3, "testtestov\0", 0xA)     = 10 0
1377/0x3ebc:      4845      64     63 close(0x3)     = 0 0
```

Фигура: Dtruss (откъс)



Фигура: Procmon (стек)

Обобщение

- Изложихме основните задачи, които решава една ОС.
- Уточнихме какво отличава ОС от библиотека за общо предназначение.
- Посочихме двата метода, чрез които ОС споделя ресурсите на машината.
- Разгледахме основната техника, използвана от ОС, за да изолират изпълнението на приложенията от хардуера.

За четене вкъщи

- Remzi H. Arpaci-Dusseau и Andrea C. Arpaci-Dusseau. *Introduction to Operating Systems*. `https://pages.cs.wisc.edu/~remzi/OSTEP/intro.pdf`. Chapter from Operating Systems: Three Easy Pieces, Version 1.10. 2018
- Remzi H. Arpaci-Dusseau и Andrea C. Arpaci-Dusseau. *Mechanism: Limited Direct Execution*. `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-mechanisms.pdf`. Chapter from Operating Systems: Three Easy Pieces, Version 1.10. 2018
- David Drysdale. *Anatomy of a system call, part 1*. `https://lwn.net/Articles/604287/`. Article from LWN.net. 2014

Край